Solution: Dutch National Flag Problem

Problem Statement

Given an array containing 0s, 1s and 2s, sort the array in-place. You should treat numbers of the array as objects, hence, we can't count 0s, 1s, and 2s to recreate the array.

The flag of the Netherlands consists of three colors: red, white and blue; and since our input array also consists of three different numbers that is why it is called **Dutch National Flag problem**.

Example 1:

```
Input: [1, 0, 2, 1, 0]
Output: [0 0 1 1 2]
```

Example 2:

```
Input: [2, 2, 0, 1, 2, 0]
Output: [0 0 1 2 2 2 ]
```

Solution

The brute force solution will be to use an in-place sorting algorithm like Heapsort which will take $O(N * logN)$. Can we do better than this? Is it possible to sort the array in one iteration?

We can use a **Two Pointers** approach while iterating through the array. Let's say the two pointers are called `low` and `high` which are pointing to the first and the last element of the array respectively. So while iterating, we will move all 0s before `low` and all 2s after `high` so that in the end, all 1s will be between `low` and `high`. In the end, all 0s are on the left, all 1s are in the middle, and all 2s are on the right.

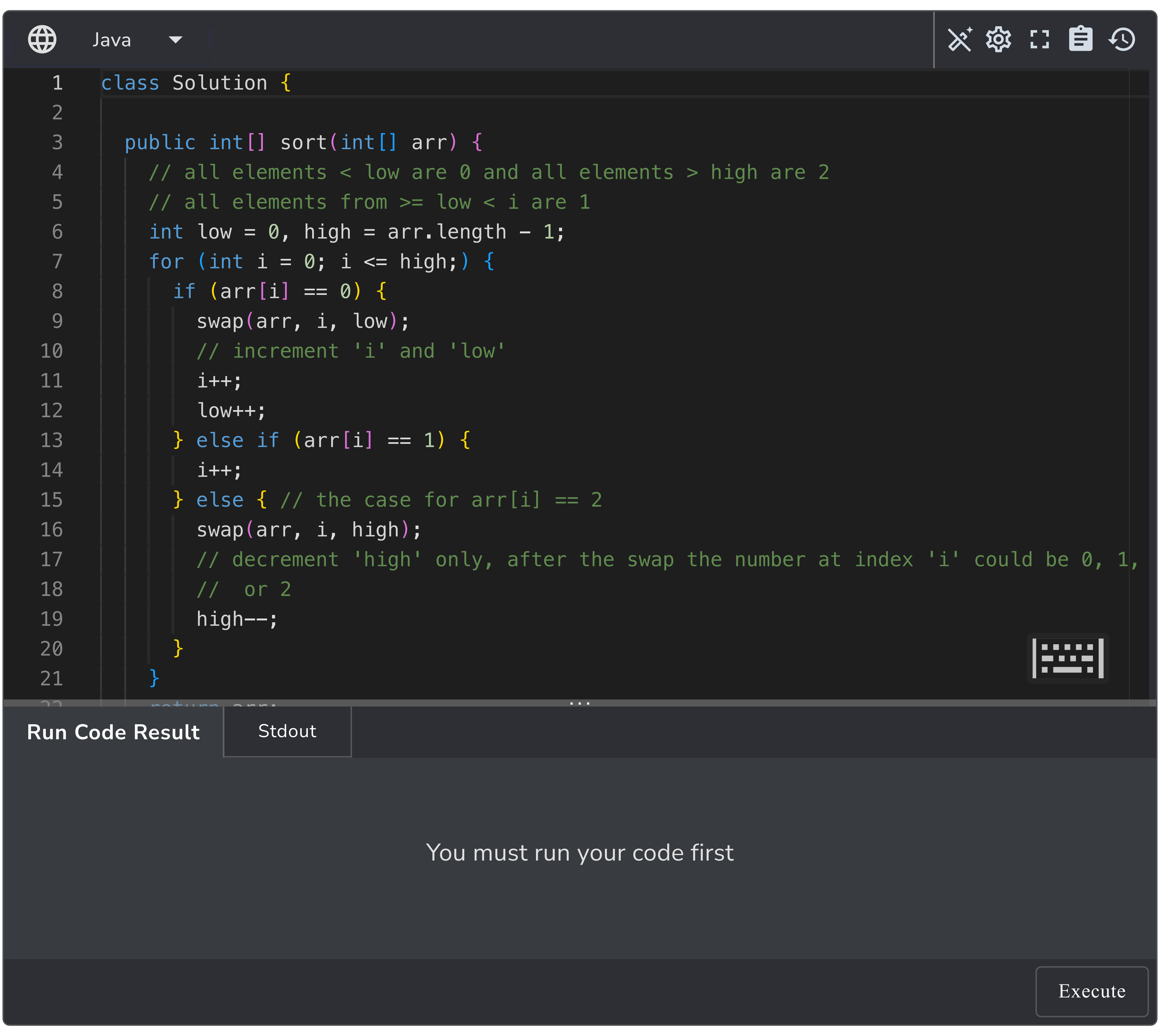
Here's a detailed walkthrough of the algorithm:

1. We start by initializing three variables: `low` to 0, `high` to the last index of the array, and `i` also to 0. `Low` is meant to keep track of the latest position where a 0 should be placed, `high` is meant to keep track of the latest position where a 2 should be placed, and `i` is used to iterate through the array.
2. We then start a loop that continues as long as `i` is less than or equal to `high`. In each iteration of the loop, we check the value of the array at the index `i`.
3. If the value is 0, we swap the values at the indices `i` and `low`. We then increment both `i` and `low`, since we know that the new element at `low` is 0 (sorted in its correct place) and we can move to the next index.
4. If the value is 1, we do nothing other than increment `i`. This is because we want 1s to be in the middle of the array.
5. If the value is 2, we swap the values at `i` and `high`. However, after the swap, we only decrement `high` without incrementing `i`. This is because the new value at index `i` (after the swap) could be 0, 1 or 2, and we need to check this value again in the next iteration.
6. The `swap` function simply switches the values at two given indices in the array.
7. The process continues until `i` is greater than `high`, at which point every element in the array has been checked and placed in its correct position. Hence, the array is now sorted.

Let's consider the example 1 and understand it via visual representation.

Code

Here is what our algorithm will look like:



```java
class Solution {

  public int[] sort(int[] arr) {
    // all elements < low are 0 and all elements > high are 2
    // all elements from >= low < i are 1
    int low = 0, high = arr.length - 1;
    for (int i = 0; i <= high;) {
      if (arr[i] == 0) {
        swap(arr, i, low);
        // increment 'i' and 'low'
        i++;
        low++;
      } else if (arr[i] == 1) {
        i++;
      } else { // the case for arr[i] == 2
        swap(arr, i, high);
        // decrement 'high' only, after the swap the number at index 'i' could be 0, 1,
        //  or 2
        high--;
      }
    }
```

You must run your code first

Time Complexity

The time complexity of the above algorithm will be O(N) as we are iterating the input array only once.

Space Complexity

The algorithm runs in constant space O(1).

Please wait! Loading...

← Previous
Dutch National Flag Problem (Medium)

Next →
Problem Challenge 1: Quadruple Sum To Target (Medium)

☐ Mark as Completed